Carpeta De Campo Proyecto Logística

Xiomara Chanel Ureña Antezana

Escuela técnica 32 distrito escolar 14 “Gral José de Sán Martín”

Tecnicatura en computación

Profesores: Nahuel, Acosta -Gabriel, Abello Tuinyla

Buenos Aires, Argentina

2026

INDICE CARPETA DE CAMPO

PÁG 3: Todo las tareas realizadas en Junio

PÁG 4-6: Todo lo realizado en Julio

PÁG 7-14: Todo lo realizado en Agosto

PÁG 14-29: Todo lo realizado en Septiembre

PÁG 29-41: Todo lo realizado en Octubre

PÁG 41-43: Todo lo realizado en Noviembre

24/06/ 2026

En el día de la fecha iniciamos el recorrido del proyecto interareal. Debido a mi ausencia, no pude estar presente en la charla de introducción que dieron los profesores Nahuel y Gabriel, en la cual explicaron la dinámica de trabajo y los objetivos generales del proyecto, tampoco pude presenciar el sorteo para la conformación de grupos de proyecto y el sorteo del proyecto correspondiente.

Luego de la charla y los sorteos, mis compañeros se pusieron en contacto conmigo para explicarme de qué va el proyecto. Me añadieron al grupo de WhatsApp creado por Benjamin y me resumieron un poco sobre la charla que tuvieron.

El mismo día propusimos preguntas para el relevamiento y decidimos realizar una búsqueda de personas relacionadas al sector para poder entrevistarlas. Cada uno buscó por su parte, pero por mi parte no pude conseguir a nadie. En un principio la entrevista iba a ser a Mercado Libre y la iba a realizar Golfard, pero no se pudo realizar. La terminó llevando a cabo mi compañero Benjamin.

06/07/ 2026

El 6 de julio en el horario de laboratorio nos juntamos todos los integrantes del grupo y establecimos las clases que iban a ser determinadas en el sistema de Java, quedando las clases:

Clase padre: Persona

Clases hijas: Cliente, Proveedor, Transportista, Courier, Analista, Seguimiento

Clase normal: Paquete

Clase Padre: Producto

Clases hijas: Tecnológico, decoramiento, Indumentaria, Accesorios

Clase normal: Envío

Clase Padre: Vehiculo

Clases hijas: Avión, Barco,Tren, Camión

Clase normal : Empresa

Clase normal: Empresas adheridas

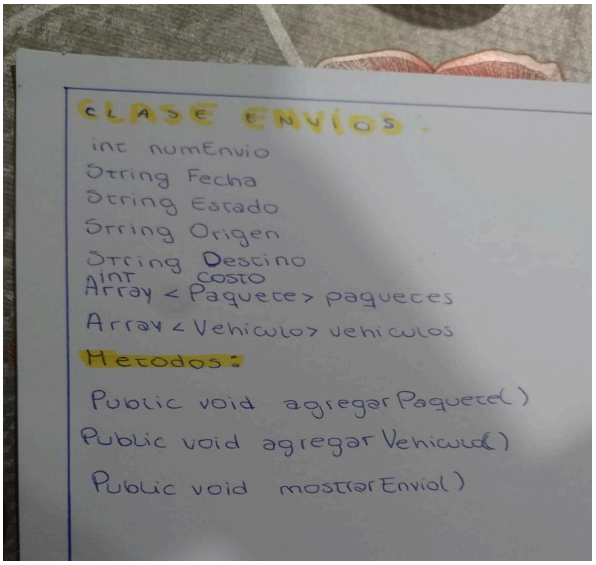
Esta misma clase una vez terminadas las clases para java nuestro líder Golfarb hizo la división de clases por cada integrante del grupo. A mi me tocó la clase envíos, tenía que crear sus respectivos métodos y atributos, pero no para programar todavia solo para el diagrama, la fecha máxima era el domingo de esa semana para tener las cosas al día y poder trabajar la semana siguiente.

11/07/ 2026

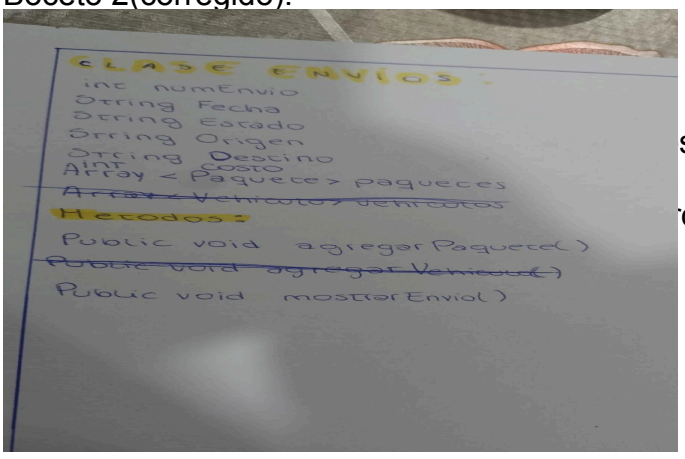
El mismo día de la fecha límite organice y realice los bocetos en papel requeridos en mi caso la clase envió con sus respectivos campos y atributos, que me la asigno el líder Golfard. Estos códigos más adelante los pasamos en limpio a la computadora en NetBeans.

Una vez terminado lo mande para verificar que esté bien. Me hicieron una corrección en vehículo ya que un paquete no puede tener varios vehículos.

Boceto 1:



Boceto 2(corregido):



s y fueron corregidos por nuestro

on la orden de pasar las clases que

```java
package ejercicio;

import java.util.ArrayList;
import javax.swing.JOptionPane;

public class Envio {

    private int numEnvio;
    private String fecha;
     private String estado;
    private String origen;
    private String destino;
    private  int costo;
    private ArrayList<Paquete> paquetes;

    public Envio(int numEnvio, String fecha, String estado, String origen, String destino, int costo) {
        this.numEnvio = numEnvio;
        this.fecha = fecha;
        this.estado = estado;
        this.origen = origen;
        this.destino = destino;
        this.costo = costo;
        this.paquetes = new ArrayList<>();
    }

    public int getNumEnvio() {
        return numEnvio;
    }

    public void setNumEnvio(int numEnvio) {
        this.numEnvio = numEnvio;
```

20/07 - 31/07 Vacaciones de invierno

05/08/2026

Finalizadas las vacaciones de invierno, el día miércoles 5 de agosto tuvimos la primera pre entrega, en la cual los profesores Nahuel Acosta y Gabriel Tuinyla nos hicieron corrección de todas las cosas que llevábamos haciendo hasta el momento. Nos fue bastante bien, exceptuando algunos detalles. Tuvimos que eliminar tablas del uml por temas de redundancia. Además de esto actualizar el diagrama de tareas que fuimos realizando en ese momento y las que venían adelante.

Mi siguiente tarea fue actualizar el diagrama de gantt, que lo hice el mismo día con mi compañero Benjamín.

06/08/2026

El 6 de agosto nos juntamos con mis compañeros en la hora de nuestro profesor Nahuel Acosta a realizar el presupuesto de la red, participamos todos en la búsqueda de los elementos que formarán parte de la infraestructura de nuestra empresa, incluyendo PCs, periféricos, routers, switches y cables, para posteriormente calcular el presupuesto final. Nuestro objetivo primordial fue reducir al mínimo el gasto económico y priorizar soluciones de bajo costo.

Los componentes implementados al listado final son las computadoras con todos sus componentes implementados a listado final son las computadoras con todos sus componentes, el switch gigabit de 24 puertos, el router cisco 2811 con 2 puertos EtherNet, cable de red ventin 2 metros de 40 gbps, el cable coaxil fino, el teléfono IP, la impresora de láser, el plotter de corte 63 cm láser y la cámara de seguridad(Imou Ranger Se Wifi Audio 360° 2mp Alarm).

Como presupuesto final nos dio: $695841416.

Realizado en hoja de cálculo de google.

Presupuesto

Archivo Editar Ver Insertar Formato Datos Herramientas Extensiones Ayuda

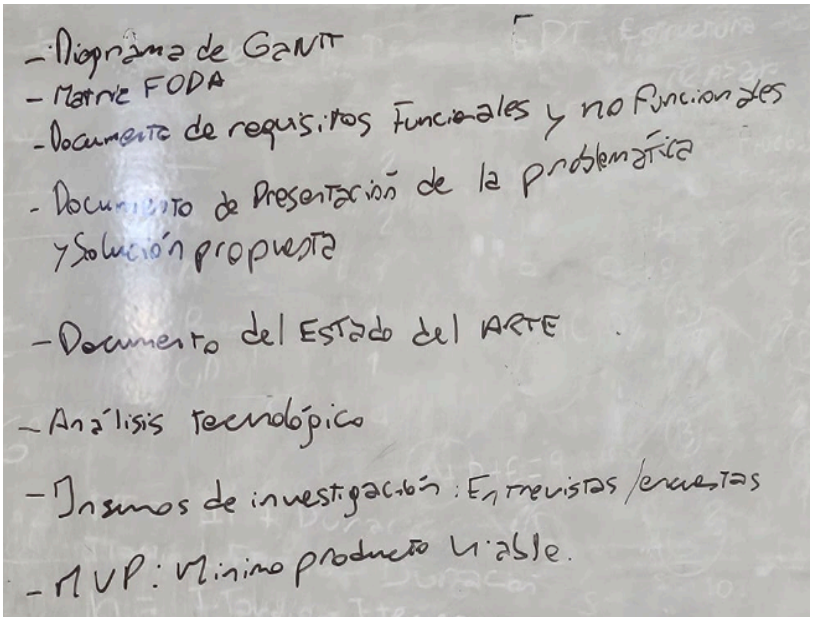
Menús 100% $ % 123 Predet... 10 B I A

E13 fx 450000

	A	B	C	D	E	F	G	H	I
1		Presupuesto Compañia de logistica							
2		Computadoras :			Unidad	Cantidad	Precio total	Total de todo	
3		Procesador:Intel core I5 4Gen						695841486	
4		Ram: 16gb DDR3							
5		GPU:Intel graphics							
6		Disco duro: SSD 256gb							
7		Windows 10							
8		Puerto Ethernet							
9		Modelo HP PRO DESK							
10		Placa: No dice							
11		Monitor 19 pulgadas							
12		Mouse y teclado							
13		Precio			450000	1200	540000000		
14		Switch							
15		Modelo Switch gigabit 24 puertos			169491	124	21016884		
16		Router							
17		modelo cisko 2811 2 puertos fast ethernet			149999	10	1499990		
18		Cables :							
19		Cable ethernet:							
20		Cable de red vention 2 metros 40gbps			11589	2611	30258879		
21		Cable coaxil fino							
22		5 metros rg6			6500	20	130000		
23		Telefono ip			68970	1200	82764000		
24		Impresora red laser			242400	20	4848000		
25		Plotter de corte 63 cm de laser			527700	2	1055400		
26		Impresora Creality Color Verde oscuro Ender 3 V3 Se			415999	27	11231973		
27		Camara De Seguridad Imou Ranger Se Wifi Audio 360° 2mp Alarm			33016	60	1980960		
28									

07/08/2026

El día viernes 7 de agosto el profesor Gabriel Tuinyla nos dio a todos los grupos un listado de las cosas que tenemos que tener hechas para la siguiente pre entrega del 14/08.



Dado el listado de las cosas que tenemos que tener hechas para la siguiente pre entrega Benjamin se encargó de dividir todas las consignas entre todos los miembros del grupo. A mi se me asignó realizar la documentación del estado de arte junto a mi compañero Nicolas Rosato.

09/08/2026

Este día con mi compañero Nicolas Rosato llevamos a cabo una investigación exhaustiva orientada al análisis de la competencia con el fin de identificar fortalezas, debilidades, ventajas y desventajas del sector logístico. El estudio se centró en las firmas Andreani, Correo Argentino y Mercado libre realizando una matriz comparativa detallada entre ambas entidades para extraer mejores prácticas aplicables a nuestro desarrollo. Él se encargó de investigar sobre Andreani y Correo Argentino, y yo de Mercado libre, después nos juntamos para ver que cosas se repetían e identificar fortalezas, debilidades, ventajas y desventajas del sector logístico.

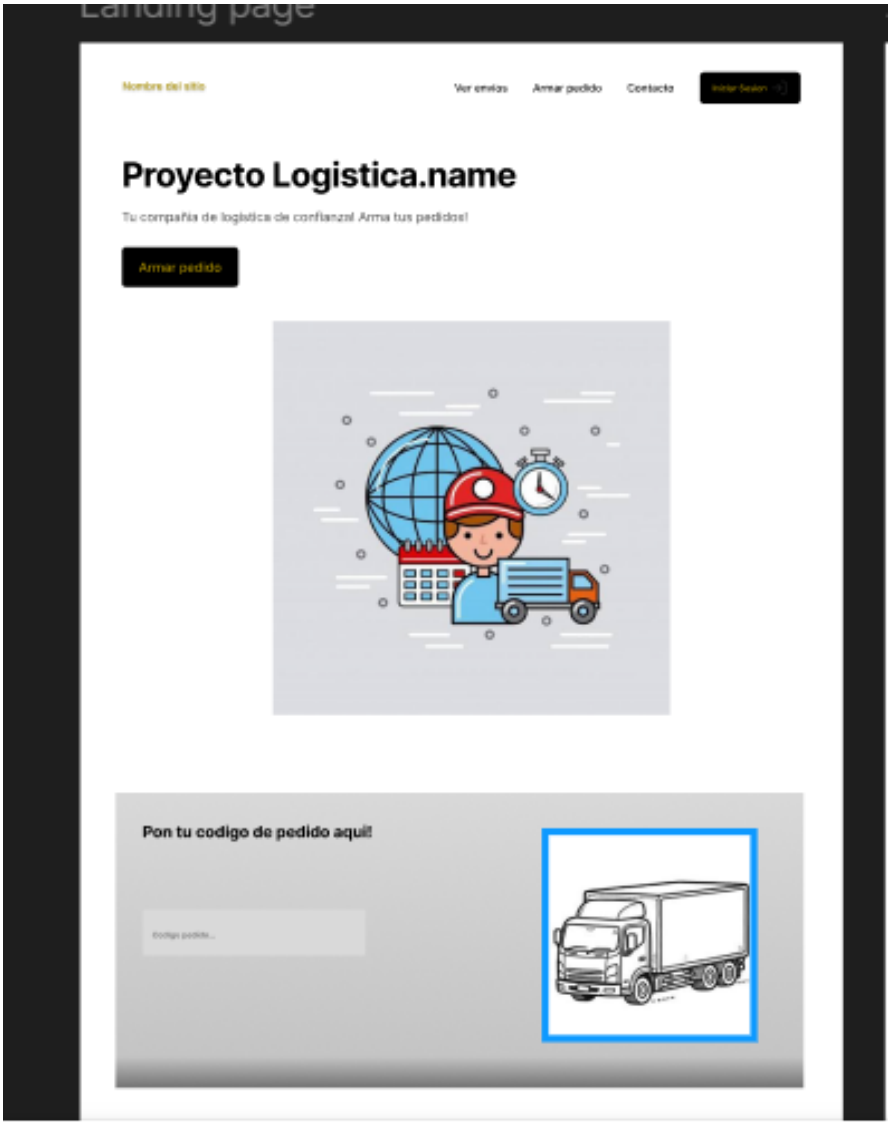
A partir del análisis de Correo Argentino, Mercado Libre y Andreani, pudimos identificar diferentes funcionalidades utilizadas actualmente, como el registro de paquetes, el seguimiento de envíos, la gestión de clientes y la organización de entregas.

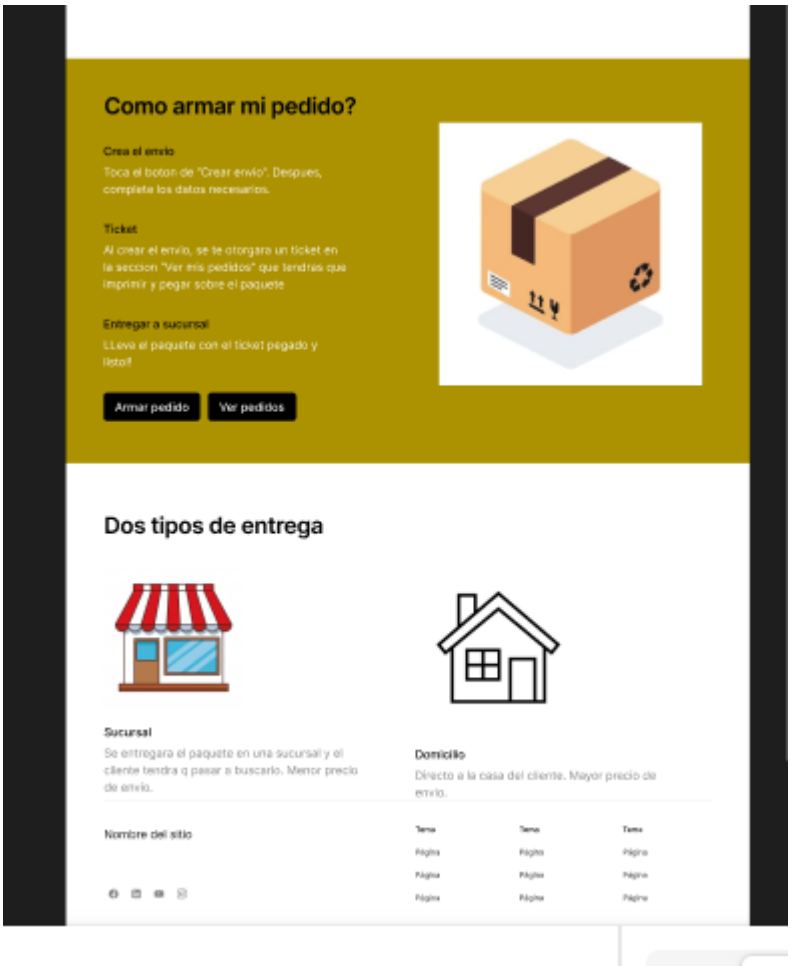
Tomando estas características como referencia, nuestro proyecto propone un sistema de logística que permite centralizar y relacionar toda la información necesaria para realizar un envío. Según nuestro diseño, se pueden registrar los clientes, empleados, empresas adheridas, productos. paquetes, vehículos y

Durante la siguiente semana mi compañero Benjamin nos asignó a Golfard y a mí realizar un boceto de lo que sería una posible versión de la página web del proyecto (el mvp).

13/08/2026

Este día con mi compañero y líder Golfard comenzamos con el Mvp, el boceto fue realizado en la página figma. Él creó el proyecto y me lo compartió por mail para que lo podamos editar al mismo tiempo. Al ser una página nueva que nunca usamos se nos complicó llegar a entender cómo funcionaba y poder terminar con el boceto. Lo único que pudimos hacer fueron estas dos páginas que voy a dejar a continuación. También pensamos como queremos que quede la página final pero el motivo que dije anteriormente no lo pudimos llevar a cabo.





Al día siguiente, tras evaluar contratiempos se procedió a una restauración de la planificación de tareas, por lo cual la responsabilidad de continuar el desarrollo del MVP fue estratégicamente reasignada para mis compañeros Golfard y Pedraza.

17/08/2026

Este día nuestro profesor Gabriel Tuinyla nos dio un listado de los requerimientos de sus materias para la siguiente pre entrega que será a fines de agosto específicamente el 30/08 .

Analisis de sistema:

- Camino crítico
- Caso de uso extendido

- Dfd ()
- Historia de uso
- Calculo de horas

Base de datos:

- Actualizar diccionario de datos
- Actualizar DER

Proyecto informático 2:

- Terminar mvp

Dados los requerimientos mi compañero Benjamin se encargó de establecer la tarea de cada uno de mis compañeros. En mi caso me tocó hacer el caso de uso extendido y el conteo de horas.

29/08/2026

Justo un día antes de tener que hacer la pre entrega, y encima el mismo día en que vencía el plazo límite, me puse a redactar el documento del caso de uso extendido. La verdad es que se me juntó todo a último momento y fue un margen de tiempo super ajustado, a mi me hubiera encantado terminar muchísimo antes para trabajar con más tranquilidad, pero se me hizo imposible porque en esa mismas fechas tenía otros exámenes que también exigen bastante tiempo.

Casos de uso extendido - Sistema de Logística

CU-01: Crear y Armar Paquete

- **Nombre:** Crear y armar un paquete
- **Actor principal:** Empleado
- **Actores secundarios:** Sistema de base de datos
- **Precondición:** El empleado debe estar autenticado. Los productos deben estar registrados y contar con stock disponible.
- **Postcondición:** El paquete queda registrado y asociado a los productos seleccionados.

Flujo principal

1) El empleado selecciona la opción "Crear nuevo paquete".
2) El sistema solicita los datos del paquete: peso, volumen y fecha de ensamble.
3) El empleado ingresa los datos solicitados.
4) El sistema muestra los productos disponibles registrados.
5) El empleado selecciona uno o varios productos para incluir en el paquete.
6) El sistema verifica que los productos seleccionados tengan stock disponible.
7) El sistema registra el paquete en la tabla **Paquetes**.
8) El sistema registra la relación entre el paquete y los productos en **Pro_pa**.
9) El sistema muestra la confirmación del paquete creado.

Flujo alternativo

5.a. Producto sin stock:
El sistema informa que el producto seleccionado no tiene stock disponible y solicita seleccionar otro producto.

3.a. Datos inválidos:
Si el peso o volsumen ingresados son inválidos, el sistema solicita corregir los datos.

Flujo de excepción

7.a. Error al registrar el paquete:
Si ocurre un error al guardar la información, el sistema informa que no fue posible crear el paquete y permite reintentar la operación.

CU-02: Registrar Envío

- **Nombre:** Registrar envío
- **Actor principal:** Empleado
- **Actores secundarios:** Sistema de base de datos
- **Precondición:** Deben existir registros de clientes, paquetes y ciudades. El empleado debe estar autenticado.

Por otro lado, la idea original era que yo también me encargara de armar el conteo

de horas del equipo. Sin embargo, por el mismo motivo y otras cosas imprevistas no pude cumplir con esta parte de la tarea. Para no retrasar al grupo mi compañero Benjamín reasigno la responsabilidad a mi compañero Agustin Zowmir, quien se hizo cargo de registrar y pasar en limpio el conteo de horas.

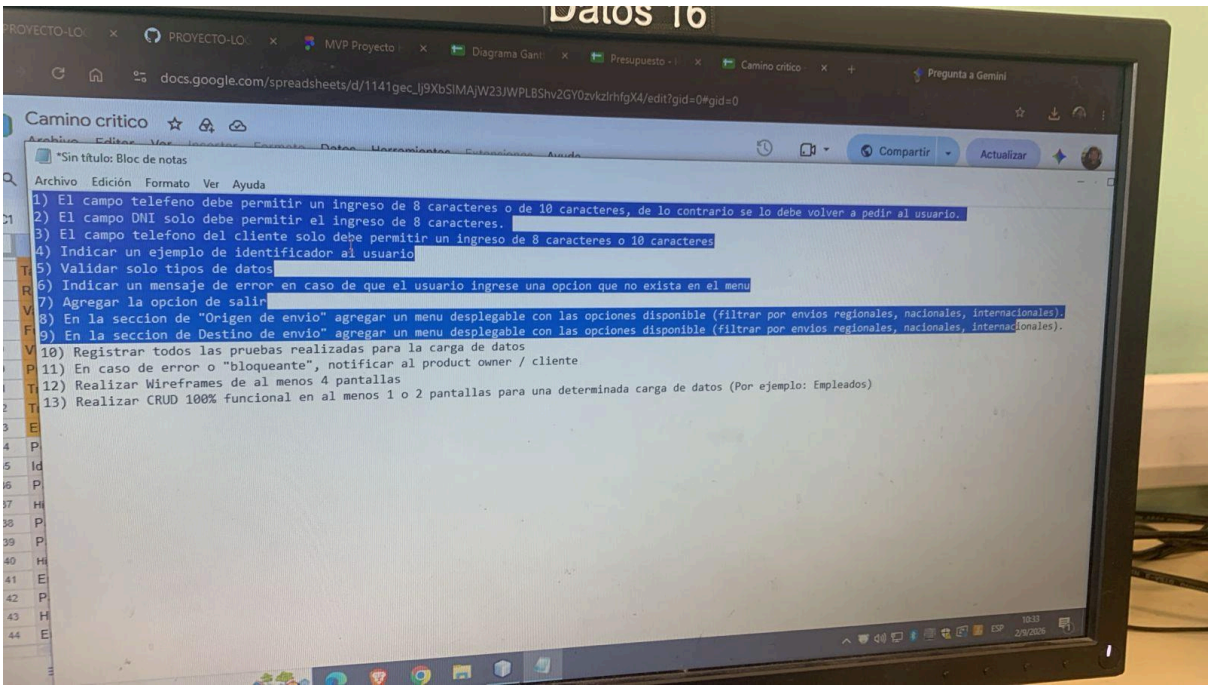
02/09/2026

En esta jornada nos tocó afrontar una nueva instancia de pre entrega para mostrar todo el avance que veníamos acumulando hasta la fecha. Esta presentación ya venía con una serie de requisitos e indicaciones previas que nos habían pedido los profesores Nahuel Acosta y Gabriel Tuinyla. La razón por la cual tuvimos que exponer en este momento se debió a que, en la fecha anterior del 31 de agosto, el tiempo no alcanzó y los profesores solamente llegaron a revisar las carpetas de los tres primeros grupos de la lista, quedando el nuestro para el 2/9. Cuando llegó nuestro turno de pasar a la corrección y nos evaluaron el material, los profesores nos marcaron varios puntos flojos y errores de debíamos solucionar:

- Con respecto al MVP: Nos comentaron que el diseño de las pantallas y la interfaz todavía se veía demasiado informal y que le faltaba una estética más pulida profesional.
- Sobre el DER (Diagrama entidad relación): Nos dijeron que había que rehacerlo directamente por completo porque visualmente era un caos, estaba muy desorganizado y encima habían quedado varias tablas repetidas que hacian ruido.
- En el diccionario de datos: Nos avisaron que había quedado desactualizado en comparación con los últimos cambios que hicimos, así que nos tocó ponernos a revisarlo para ponero al dia.

- En la parte de POO(Programación Orientada a Objetos): También nos encontraron cosas para corregir y nos dejaron apuntadas varias tareas y modificaciones que debíamos implementar en el código.

Una vez terminada la pre entrega, nos reunimos virtualmente para ver cómo dividimos el trabajo. Nuestro líder de equipo, Golfard, analizó la situación y nos asigno a Nicolas Rosato y a mí la tarea de encargarnos de todas las correcciones correspondientes al código de Java, que son justamente las que se pueden ver en la foto que adjunto acá abajo. Cabe aclarar un detalle muy importante, nos pusimos de acuerdo en que solamente íbamos a resolver los primeros 9 puntos de la lista, ya que las consignas restantes exigen usar herramientas y conceptos de programación que todavía no llegamos a ver en la materia de POO, por lo que era imposible avanzar con eso por el momento.



Este día solo llegamos a hacer los dos primeros puntos. Los cuales eran:

1) El campo teléfono debe permitir un ingreso de 8 caracteres, de lo contrario se lo debe volver a pedir al usuario.

2) El campo dni solo debe permitir el ingreso de 8 caracteres.

3) El campo de teléfono del cliente solo debe permitir un ingreso de 8 caracteres o 10 caracteres

```
}
if(dni.length()!=8){
    JOptionPane.showMessageDialog(null, "ERROR - El DNI debe tener solo 8 digitos");
    contador++;
    return;
}


// condicionales telefono
if(!telefono.matches("[0-9]+") && !telefono.equals("")){
    JOptionPane.showMessageDialog(null, "ERROR - El telefono solo puede contener numeros.");
    contador++;
    return;
}
if(telefono.length()!=11 && telefono.length()!=10){
    JOptionPane.showMessageDialog(null, "ERROR - El telefono debe contar con 10 u 11 digitos");
    contador++;
    return;
}
```

06/09/2026

El viernes 6 de septiembre nos juntamos con Nicolas para terminar de realizar lo pedido, nos quedaban pendientes 7 puntos los cuales eran:

4) Indicar un ejemplo de identificador al usuario.

5) Validar solo tipos de datos.

6) Indicar un mensaje de error en caso de que el usuario ingrese una opción que no exista en el menú.

7) Agregar la opción de salir.

8) En la sección de “Origen de envío” agregar un menú desplegable con las opciones disponibles (filtrar por envíos regionales, nacionales, internacionales).

9) En la sección de “Destino de envío” agregar un menú desplegable con las opciones disponibles (filtrar por envíos regionales, nacionales, internacionales).

Cuando llegamos al punto 8 nos dimos cuenta que lo de menú desplegables no lo sabíamos porque todavía no lo habíamos visto en clase. Nuestro compañero Benjamin por su parte investigó como armar los menús desplegables en Java, logró entender la lógica del código, se encargó de programarlo él mismo.

09/09/2026

Este dia tuvimos una charla con nuestros profesores en donde nos dieron los requisitos para la pre entrega de septiembre los cuales son:

- POO(Programación Orientada a Objetos): Todas las ventanas que vayamos a usar con CRUD funcional y contador de errores.
- Redes: Servidor FTP y Correo.
- Proyecto Informático: Pantallas hasta donde lleguemos
- Base de datos: Procedimientos.

Una vez finalizada la charla nuestro compañero Benjamin se encargó de asignarnos nuestras respectivas tareas, se nos asigno realizar tres procedures básicos (insertar, editar y eliminar) para las distintas tablas, siendo un total de 9 procedures..

La división fue dos tablas normales y una tabla intermedia por integrante. A mi se me asignaron las tablas “Reportes”, “Reportes_tickets” y “Tickets”.

- Tickets: Al crear un tickets se debe verificar que el cliente exista y no se pueden borrar tickets que están en la tabla intermedia, reportes_tickets.

- Reportes: Al crear un reporte se debe verificar que el cliente exista y no se pueden borrar reportes que están en la tabla intermedia, reportes_tickets.
- Reportes_tickets: Se debe verificar que exista ese reporte y ese ticket.

No recuerdo bien las fechas, pero durante la siguiente semana estuve realizando los procedures

Procedimientos:

Agregar Tickets:

```
DELIMITER //

CREATE PROCEDURE agregarTickett(
   IN idCliente INT,
   IN fechaN DATE,
   IN detalleN VARCHAR(100),
   IN totalN INT,
   IN metodospagoN VARCHAR(25),
   OUT mensaje VARCHAR(200)
)
BEGIN
   DECLARE v_existe INT DEFAULT 0;

   START TRANSACTION;
      SELECT COUNT(*) INTO v_existe
      FROM Clientes
      WHERE ID_Cliente = idCliente;

      IF v_existe = 0 THEN
         SET mensaje = 'Error: El cliente no existe. Transacción cancelada.';
         ROLLBACK;
      ELSE
         INSERT INTO Tickets (ID_Cliente, fecha, detalle, total, metodospago)
         VALUES (idCliente, fechaN, detalleN, totalN, metodospagoN);

         COMMIT;
         SET mensaje = 'Ticket creado con éxito';
      END IF;
END //
```

```
DELIMITER ;
```

Eliminar Tickets:

```
DELIMITER //
CREATE PROCEDURE eliminarTicket (in p_idTicket int, out mensaje varchar(100))
BEGIN
        DECLARE contador_reportesTickets int;
      START TRANSACTION;
        SELECT COUNT(reportes_ticket.ID_rt) INTO contador_reportesTickets
FROM reportes_ticket WHERE reportes_ticket.ID_Ticket=p_idTicket ;
         IF contador_reportesTickets>0 THEN
                SET mensaje='No se puede eliminar un ticket que este en la tabla
reportes tickets';
             ROLLBACK;
           ELSE
             DELETE FROM tickets
             WHERE tickets.ID_Ticket =p_idTicket ;
             COMMIT;
             SET mensaje='Se ha realizado con exito';
             END IF;
END //
DELIMITER ;
  DELIMITER //
```

Editar Ticket:

```
create procedure editar_ticket(in id_ticket int,
    in id_cliente int,
    IN fechas date,
    in detalles varchar(1000),
    in ttotal int,
    in metodoPago varchar(100),
    OUT mensaje VARCHAR(200)
)
BEGIN
        declare auxiliar varchar(100);
      start TRANSACTION;
      select tickets.detalle into auxiliar from tickets where tickets.ID_Ticket =id_ticket ;
         if auxiliar is null THEN
                set mensaje='No existe el ticket ';
            rollback;
          else
                if id_cliente  is not null THEN
```

```
                update tickets
                set ID_Cliente = id_cliente
                where ID_Ticket=id_ticket;
                end if;
              if fechas  is not null THEN
                update tickets
                set fecha=fechas
                where ID_Ticket=id_ticket;
                end if;
              if detalles is not null THEN
                update tickets
                set detalle =detalles
                where  ID_Ticket=id_ticket;
                end if;
               if ttotal is not null THEN
                update tickets
                set total=ttotal
                where ID_Ticket=id_ticket;
                end if;
               if metodoPago is not null THEN
                update tickets
                set metodosPago=metodoPago
                where ID_Ticket=id_ticket;
                end if;
                set mensaje='Se ha encontrado el ticket, se editaron los campos
proporcionados correctamente';
                commit;
                end if;
end //
delimiter ;
```

Agregar Reportes:

```
DELIMITER //
create procedure agregar_reportes(in detallee varchar(1000), out mensaje
varchar(100))
begin
   DECLARE EXIT HANDLER FOR SQLEXCEPTION
   begin
      SET mensaje = "Error. Operacion cancelada";
      rollback;
   end;
   start transaction;
   insert into Reportes( detalle)
```

```
    values (detallee);
    commit;
    set mensaje = "Reporte registrado correctamente.";
end //
DELIMITER ;
```

Eliminar Reportes:

```
DELIMITER //

CREATE PROCEDURE eliminarReporte(
    IN p_idReports INT,
    OUT mensaje VARCHAR(100)
)
BEGIN
    DECLARE contador_reportesTickets INT DEFAULT 0;

    START TRANSACTION;
        SELECT COUNT(*)
        INTO contador_reportesTickets
        FROM reportes_ticket
        WHERE reportes_ticket.ID_Reporte = p_idReports;

        IF contador_reportesTickets > 0 THEN
            SET mensaje = 'No se puede eliminar un reporte que este en la tabla reportes
tickets';
            ROLLBACK;
        ELSE
            DELETE FROM reportes
            WHERE reportes.ID_Reporte = p_idReports;

            SET mensaje = 'Se ha realizado con exito';
            COMMIT;
        END IF;
END //

DELIMITER ;
```

Editar Reportes:

```
 DELIMITER //

CREATE PROCEDURE editar_reporte(
    IN id_Reporte INT,
    IN detallee VARCHAR(1000),
    OUT mensaje VARCHAR(200)
)
```

```
BEGIN
   DECLARE existe_reporte INT DEFAULT 0;

   START TRANSACTION;
      SELECT COUNT(*) INTO existe_reporte
      FROM reportes
      WHERE reportes.ID_Reporte = id_Reporte;

      IF existe_reporte = 0 THEN
         SET mensaje = 'No existe el reporte';
         ROLLBACK;
      ELSE
         IF detallee IS NOT NULL THEN
            UPDATE reportes
            SET detalle = detallee
            WHERE reportes.ID_Reporte = id_Reporte;
         END IF;

         SET mensaje = 'Se ha encontrado el reporte, se editaron los campos
proporcionados correctamente';
         COMMIT;
      END IF;
END //

DELIMITER ;
```

Agregar Reportes_ticket:

```
DELIMITER //
create procedure agregarReportes_Ticket(in idReporte int, in idTicket int, out mensaje
varchar(75))
begin
declare existe_Reporte int default null;
declare existe_Ticket int default null;

select ID_Reporte into existe_Reporte from reportes
where ID_Reporte = idReporte;
select ID_Ticket into existe_Ticket from tickets
where ID_Ticket = idTicket;

if existe_Reporte is null or existe_Ticket is null then
set mensaje = "Error. El ticket o reporte no existe.";
else
insert into reportes_ticket(ID_Reporte, ID_Ticket)
values(idReporte, idTicket);
set mensaje = "Union Reportes-Ticket registrada correctamente.";
end if;
end //
```

```
DELIMITER ;
```

Eliminar Reportes_ticket:

```
DELIMITER //
create procedure eliminar_ReportesTicket (in id_ReporteTicket int, out mensaje varchar(75))
begin
declare existe_ReporteTicket int default null;
select ID_rt  into existe_ReporteTicket from reportes_ticket
where ID_rt = id_ReporteTicket;

if existe_ReporteTicket is null then
set mensaje = "Error. La union Reportes_ticket ingresada no existe.";
rollback;
else
        delete from reportes_ticket
   where ID_rt = id_ReporteTicket;
   set mensaje = "Union Reportes_ticket eliminada correctamente.";
end if;
end //
DELIMITER ;
```

Editar Reportes_ticket:

```
DELIMITER //
create procedure editar_ReporteTicket (in id_reporteTicket int, in idReporte int, in idTicket
int, out mensaje varchar (75))
begin
declare existe_reporteTicket int default null;

select ID_rt into existe_reporteTicket from reportes_ticket
where ID_rt = id_reporteTicket;

if existe_reporteTicket is null then
set mensaje = "Error. La union Reporte_ticket ingresada no existe.";
rollback;
else
        if idReporte is not null then
                update reportes_ticket
      set ID_Reporte = idReporte
      where ID_rt = id_reporteTicket;
        end if;
        if idTicket is not null then
                update reportes_ticket
      set ID_Ticket = idTicket
      where ID_rt = id_reporteTicket;
        end if;
```

```
set mensaje = "Editado correctamente.";
commit;
end if;
end //
DELIMITER ;
```

Para realizar estos procedures use los conocimientos que tengo y si me olvidaba de algo consultaba en trabajos realizados por mi durante el año en la materia de base de datos.

17/09/2026

Una vez que todos los integrantes del grupo finalizaron con sus procedures y fueron corroborados por nosotros mismos, Benjamin nos asignó que cada integrante debía realizar cincuenta inserts por cada tabla, de las que cada uno habia realizado los procedures.
Realicé mis 150 insterts correspondientes a las tablas “Tickets”(50), “Reportes”(50) y Reportes_ticket(50) durante esa semana.

23/09/2026

El día miércoles 23, Benjamin Saragusti nos asigno a mis compañeros Nicolas Rosato y Santiago Pedraza a realizar modificaciones al código de JAVA en el que él había estado trabajando durante la semana. La parte nos asignó corregir era la correspondiente a las primeras versiones de las ventanas interactivas JFrame correspondientes a los CRUD (sin contar aún con la conexión a la base de datos). Las modificaciones consistían principalmente en agregar validaciones básicas en los campos a completar de las tablas, por ejemplo, verificar que los campos no estén vacíos al intentar agregar algún elemento. Además, tuvimos que agregar un

contador el cual, al detectar diez veces un error al querer realizar alguna tarea en una misma ventana, cerraba el programa.
Al principio, nuestras validaciones no funcionaron como correspondía por lo que Golfarb y Saragusti verificaron el código y encontraron una solución.

30/09/2026

Este día tuvimos una pre entrega, el balance general que nos dieron los profesores fue bastante positivo dentro de todo. Nos comentaron que el proyecto venía relativamente bien encaminado, pero nos llamaron la atención sobre un par de cosas.

Lo primero fue que teníamos que arrancar si o si con el armado y diseño del sitio web en html para empezar a darle forma a la interfaz en el navegador, por otro lado a la hora de revisar la base de datos, el profesor Gabriel Tuinyla nos marcó que la cantidad de stored procedures que habíamos programado hasta el momento era baja, no alcanzaba únicamente con tener los procedimientos básicos de “Agregar”, “Borrar” y “Editar” registros, teníamos que hacer entre 10 y 15 procedures específicos para cada una de las tablas. Finalmente, para cerrar con las tareas que teníamos que hacer, los profesores nos dijeron que empezáramos con la programación de la conectividad para enlazar el código de Java con la base de datos Mysql, esto era fundamental para que los datos que cargaramos en la interfaz empezaran a guardarse de verdad y el sistema funcionara.

30/09/2026 - 06/10/2026

Durante estos días me encargue de realizar los procedimientos para llegar a la cantidad mínima solicitada, de las mismas tablas a las cuales les hice los primeros procedures previos a la entrega de septiembre(“Tickets”, “Reportes” y “Reportes_ticket”).

Ticket:

Eliminar tickets (usando id cliente)

```
DELIMITER //

CREATE PROCEDURE eliminarTicket2 (in p_idTicket int)
BEGIN
	delete  from Tickets where ID_Cliente=p_idTicket;
END //
DELIMITER ;
```

Buscar tickets (por id cliente)

```
DELIMITER //
CREATE PROCEDURE ticketes_cliente(in t_id_cliente int)
BEGIN
	select * from tickets where tickets.ID_Cliente=t_id_cliente;
END //
DELIMITER ;

 call ticketes_cliente(2);
```

Mostrar tickets

```
delimiter //
create procedure mostrar_tickets()
	BEGIN
	select * from tickets;
END //
DELIMITER ;
```

Mostrar tickets donde el pago sea en efectivo(este mismo procedimiento lo hice con Mercado Pago, Debito, Credito y Transferencia, lo pongo aca ya que es el mismo código solo cambia WHERE metodospago = "Efectivo"; **)**

```
DELIMITER //
CREATE PROCEDURE metodoPago_efectivo()
BEGIN
 SELECT * FROM tickets WHERE metodospago = "Efectivo";
END //
DELIMITER ;
```

Mostrar tickets donde el total sea menor a 50000

```
DELIMITER //
CREATE PROCEDURE total_menor50000()
BEGIN
  SELECT * FROM tickets where total < 50000;
END //
DELIMITER ;
```

Al sumar estos 9 procedimientos nuevos que hice a los 3 que ya veníamos teniendo listos desde antes (que eran los clásicos de “agregar”, “editar” y “eliminar”), logramos alcanzar un total de 12 procedimientos almacenados asignados a la tabla tickets. Con esto ya dejamos esa parte super completa y cumpliendo con lo que habían pedido los profesores.

Después de terminar eso, me puse a realizar el mismo trabajo con la tabla de reportes. La diferencia acá fue que, al ser una tabla mucho más simple que sólo contiene el campo de id y el detalle, no se podía meter tanta lógica, así que en este caso diseñe solamente 2 procedimientos nuevos. Si sumamos estos 2 a los 3 básicos que ya estaban hechos (“agregar”, “editar” y “eliminar”), nos quedó un total de 5 procedimientos para la tabla de reportes. Para terminar con todos los procedimientos, pasó exactamente lo mismo cuando me tocó trabajar con la tabla reportesTicket. Como compartía una

estructura similar y sencilla, le sume 2 procedimientos más a los 3 que ya estaban hechos antes, dejando un total de 5 para esta última tabla.

Reportes:

Buscar Reportes:

```
DELIMITER //
CREATE PROCEDURE buscar_reportess(in id int)
BEGIN
  select * from reportes WHERE ID_Reporte = id;
END //
DELIMITER ;
```

Mostrar Reportes:

```
delimiter //
create procedure mostrar_reportes()
	BEGIN
	select * from reportes;
END //
DELIMITER ;
```

Reportes_Ticket:

Buscar Reportes_Ticket por id:

```
DELIMITER //
CREATE PROCEDURE buscar_reportesTickets(in id int)
BEGIN
  select * from reportes_ticket WHERE ID_rt = id;
END //
DELIMITER ;
```

Mostrar Reportes_Ticket:

```
delimiter //
create procedure mostrar_reportesTickets()
	BEGIN
	select * from reportes_ticket;
```

END //
DELIMITER ;

07/10/2026

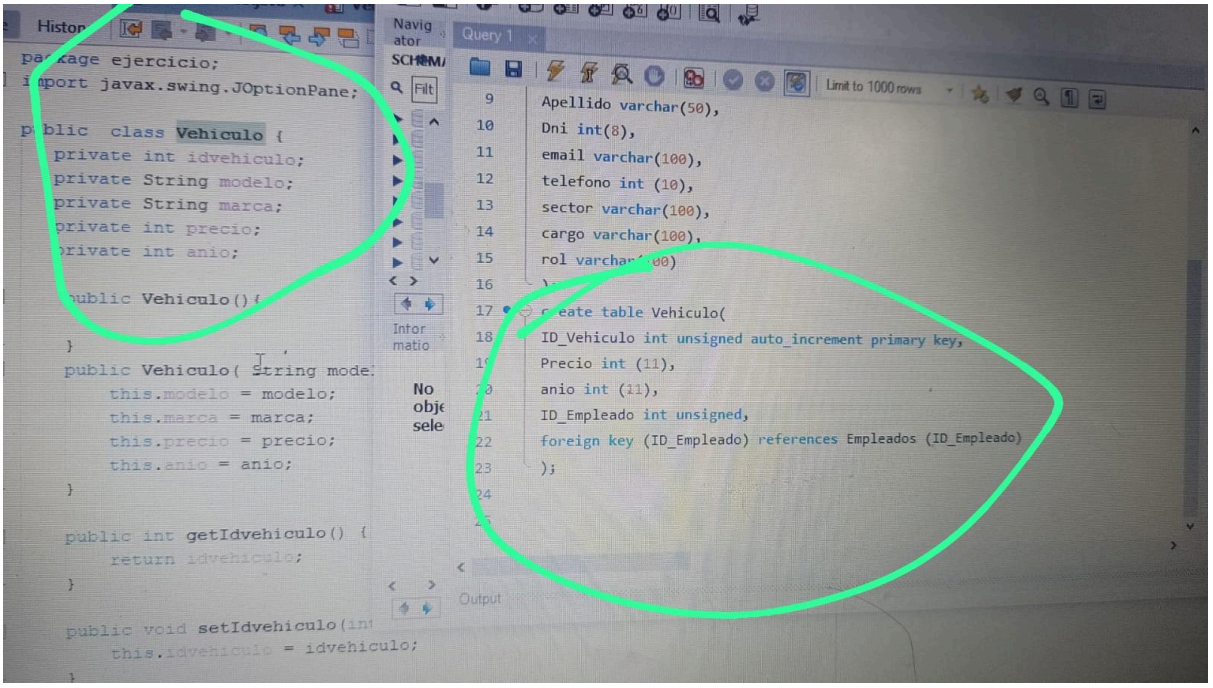
Durante esta jornada, nuestro líder de equipo Golfard, llevó a cabo la distribución de las tareas de desarrollo en Java, asignó a cada integrante del grupo la responsabilidad de programar el diseño de una ventana. Un detalle fundamental de esta etapa es que aca ya estábamos implementando la conexión real y directa con la base de datos por lo que el objetivo principal era lograr que cada ventana se acoplara y adaptara a la perfección con las tablas correspondientes y con los parámetros de entrada que ya teníamos definidos en la base de datos.

En mi caso, me tocó el desarrollo completo de la ventana Vehículo. Las funcionalidades que tenía que tener esta ventana son las de agregar, buscar, mostrar, editar y borrar.

08/10/2026 - 12/10/2026

Durante estos días, estuve metida de lleno con el desarrollo y la programación de la ventana que me había tocado, que era la de Vehículos. Sabiendo que venía bastante complicada con los tiempos y que el margen de entrega era ajustado, decidí arrancar lo antes posible para intentar adelantar. Sin embargo, a pesar del esfuerzo, se me presentaron varias complicaciones en el medio que me impidieron terminar todo a tiempo. La fecha límite original que teníamos pautada para la entrega era el domingo, pero debido a estos contratiempos, no me quedó otra opción que retrasarme un poco y terminar entregando todo el lunes 12.

El principal conflicto que tuve apareció apenas arranque con el código. Al ponerme a revisar en detalle para empezar a enlazar las pantallas, me di cuenta de que los atributos que definió alguno de mis compañeros para la clase “Vehículos” en Java no coinciden para nada con los campos que tenía la tabla de “vehículos” en la base de datos de MySQL. Había diferencias en los nombres y en la cantidad de datos, lo que hacía imposible que la información viaje de un lado al otro.



Para resolver esto me comunique con mis compañeros Golfarb y Benjamin. Al principio yo pensaba que tenía que modificar la base de datos, ya que faltaban los atributos modelo y marca pero no, mis compañeros me dijeron que estos dos atributos estaban agregados al final de todas las tablas con el alter table. Lo único que tuve que agregar fue un atributo en Java, IdEmpleado ya que no estaba, como consecuencia también tuve que modificar el constructor. Otra cosa que también tuve que agregar fue el constructor vacío para después poder usarlo en la clase vehiculosDAO.

Una vez resuelto eso pude avanzar con la ventana VehiculoDAO quedando así:

```
package ejercicio;
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;

public class VehiculoDAO {
    public void insertar(Vehiculo v)throws SQLException {
        String consulta_sql = "CALL crear_vehiculo(?, ?, ?, ?, ?, ?)";
        try{
            Connection conexion = Conexion.conectar();
            CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

            sentencia_sql.setInt(1, v.getPrecio());
            sentencia_sql.setInt(2, v.getAnio());
            sentencia_sql.setInt(3, v.getIdEmpleado());
            sentencia_sql.setString(4, v.getModelo());
            sentencia_sql.setString(5, v.getMarca());
            sentencia_sql.registerOutParameter(6, java.sql.Types.VARCHAR);
            sentencia_sql.executeUpdate();
            String mensajeRespuesta = sentencia_sql.getString(6);
            System.out.println("Respuesta del servidor: " + mensajeRespuesta);
```

```
            System.out.println("Vehiculo guardado correctamente");
        }catch(SQLException ex){
            System.out.println("ERROR "+ ex.getMessage());
        }
    }

    public ArrayList <Vehiculo> listar(){
        ArrayList <Vehiculo> lista_vehiculo = new ArrayList<>();
        String consulta_sql="call mostrar_vehiculos()";
        try{
            Connection conexion = Conexion.conectar();
            CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
            ResultSet resultado=sentencia_sql.executeQuery();
            while (resultado.next()){
                Vehiculo v = new Vehiculo();

                v.setIdvehiculo(resultado.getInt("ID_Vehiculo"));
                v.setPrecio(resultado.getInt("Precio"));
                v.setAnio(resultado.getInt("anio"));
                v.setIdEmpleado(resultado.getInt("idEmpleado"));
                v.setModelo(resultado.getString("Modelo"));
                v.setMarca(resultado.getString("Marca"));

                lista_vehiculo.add(v);
                System.out.println("~~~~~~~~~~~~~~~~~~");
```

```
        }
    }catch (SQLException ex){
        System.out.println("ERROR "+ ex.getMessage());
    }
    return lista_vehiculo;
}

public void actualizar (Vehiculo v){
     String consulta_sql = "call editar_vehiculos (?, ?, ?, ?, ?, ?, ?)";
    try {
        Connection conexion = Conexion.conectar();
        CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

        sentencia_sql.setInt(1, v.getIdvehiculo());
        sentencia_sql.setInt(2, v.getPrecio());
        sentencia_sql.setInt(3, v.getAnio());
        sentencia_sql.setInt(4, v.getIdEmpleado());
        sentencia_sql.setString(5, v.getModelo());
        sentencia_sql.setString(6, v.getMarca());

        sentencia_sql.registerOutParameter(7, java.sql.Types.VARCHAR);
        sentencia_sql.executeUpdate();
        String respuesta = sentencia_sql.getString(7);
        System.out.println("Respuesta de la BD: " + respuesta);
        System.out.println("Datos actualizados exitoamnete");
```

```
    }catch (SQLException ex){
        System.out.println("ERROR "+ ex.getMessage());
    }
}

public Vehiculo buscar(int id) {
String consulta_sql = "call buscar_vehiculo(?)";
Connection conexion = Conexion.conectar();
try {
    CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);
    sentencia_sql.setInt(1, id);
    ResultSet resultado = sentencia_sql.executeQuery();
    if (resultado.next()) {
        Vehiculo v =new Vehiculo();
            v.setIdvehiculo(resultado.getInt("ID_Vehiculo"));
            v.setPrecio(resultado.getInt("Precio"));
            v.setAnio(resultado.getInt("anio"));
            v.setIdEmpleado(resultado.getInt("idEmpleado"));
            v.setModelo(resultado.getString("Modelo"));
            v.setMarca(resultado.getString("Marca"));

        System.out.println("------------------");
        return v;
    } else {
```

```
            System.out.println("No se encontró ningún vehiculo con el ID: " + id);
        }
    } catch (SQLException ex) {
        System.out.println("ERROR " + ex.getMessage());
    }
    return null;
}
    public void eliminar(int id) {
    String consulta_sql = "call eliminar_vehiculo (?, ?)";
    try {
        Connection conexion = Conexion.conectar();
        CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);

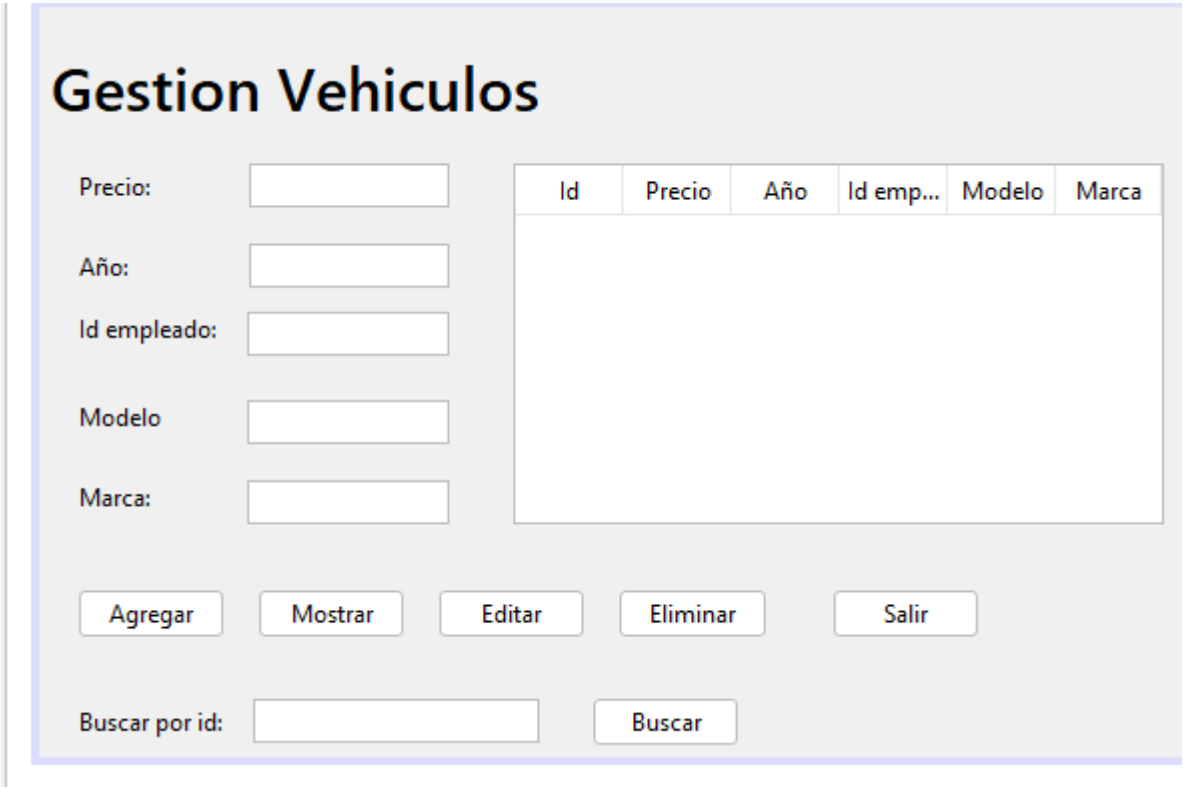
        sentencia_sql.setInt(1, id);
        sentencia_sql.registerOutParameter(2, java.sql.Types.VARCHAR);
        sentencia_sql.executeUpdate();

        String respuesta = sentencia_sql.getString(2);
        System.out.println("Respuesta de la BD: " + respuesta);
        System.out.println("Vehiculo eliminado");

    } catch (SQLException ex) {
        System.out.println("ERROR: " + ex.getMessage());
    }
}
```

}

Después continúe haciendo la ventana y desarrollando el código de cada uno de los botones(no lo pongo ya que es muy largo). La ventana me quedo asi:



Una vez terminado todo se lo mandé a mi compañero Benjamin para que lo revisara, el código estaba bastante bien, pero tuve algunas correcciones. Una de estas fue borrar el main de la ventana que me olvide por completo y otra fue que me olvide de agregar el método de actualizar tabla, ya que mi código solamente actualizaba la tabla cuando tocaba mostrar.

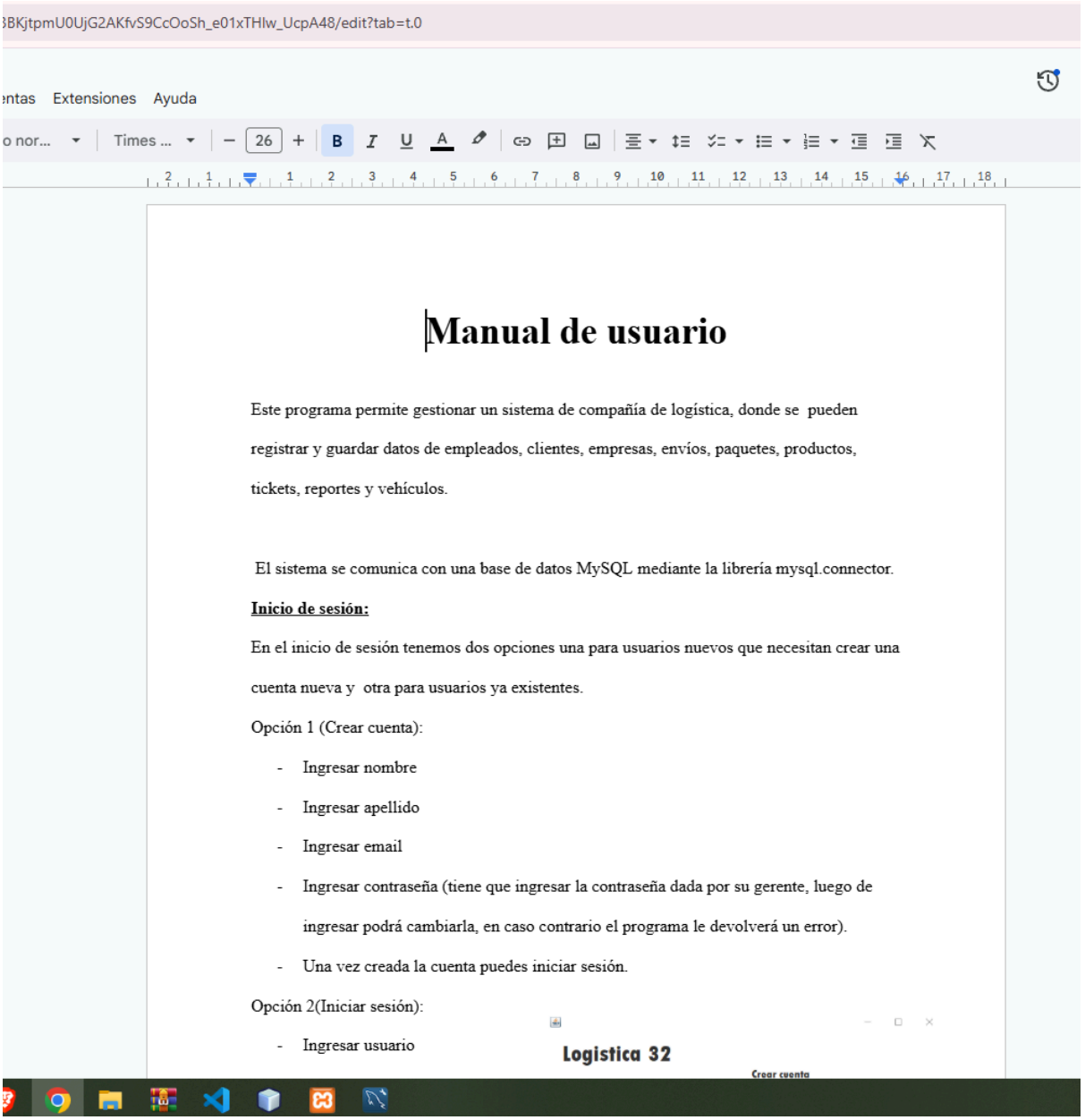
13/10/2026

Ese día Benjamin se encargó de armar el reparto de las pantallas que nos tocaba diseñadas en HTML, con el objetivo de empezar a conectarlas de una vez por todas al servidor web. En la distribución de tareas , a mi me asignaron la

responsabilidad de armar dos páginas, “Registrar productos” y ”Consultar Productos”.
La primera pantalla, la de registro, la tengo que diseñar pensando en un formulario limpio donde el usuario pueda cargar toda la información.
La segunda pantalla, la de consulta, viene con un poco más de complejidad y trabajo de lógica de mi parte. No se trata solo de mostrar el listado de lo que hay en stock, sino que dentro de esa misma página tengo que meter los botones y las funcionalidades para poder “Editar” y “Eliminar” cualquier producto directamente desde ahí.

14/10/2026 (no recuerdo bien las fechas)

No recuerdo bien las fechas pero durante las siguientes semanas estuve realizando varias cosas, lo primero que realicé fue el manual de usuario, lo hice con un poco de ayuda de mi compañero Benjamin Saragusti. Estuve dos dias con esto ya que también tenía que hacer otro proyecto para la materia Proyecto Informático 2.



Otras de las cosas que estuve realizando estas semanas fueron los triggers que me faltaban y el diseño (css) de las paginas que me tocaron (“mostrarProductos”, “registrarProductos”).

TRIGGERS XIOMARA:

Asigna automáticamente la fecha del ticket si esta es nula.

```
DELIMITER //

CREATE TRIGGER ticketFecha
BEFORE INSERT ON Tickets
FOR EACH ROW
BEGIN
   IF NEW.fecha IS NULL THEN
      SET NEW.fecha = CURDATE();
   END IF;


   END IF;
END//

DELIMITER ;
```

antes de insertar verifica que el monto total no sea negativo.

```
CREATE TRIGGER montoNegativo
```

```
BEFORE INSERT ON Tickets
FOR EACH ROW
BEGIN
   IF NEW.total < 0 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Error: El total del ticket no puede ser un valor negativo.';
```

```
      SET MESSAGE_TEXT = 'Error: El total del ticket no puede ser un valor negativo.';
   END IF;
END//

DELIMITER ;
```

Limpiar la tabla intermedia al eliminar un ticket

```
DELIMITER//
CREATE TRIGGER limpiarTabla
AFTER DELETE ON Tickets
FOR EACH ROW
BEGIN
   DELETE FROM Reportes_Ticket WHERE ID_Ticket = OLD.ID_Ticket;
END//

DELIMITER ;
```

Antes de insertar valida que ambos id existan(ticket y reporte)

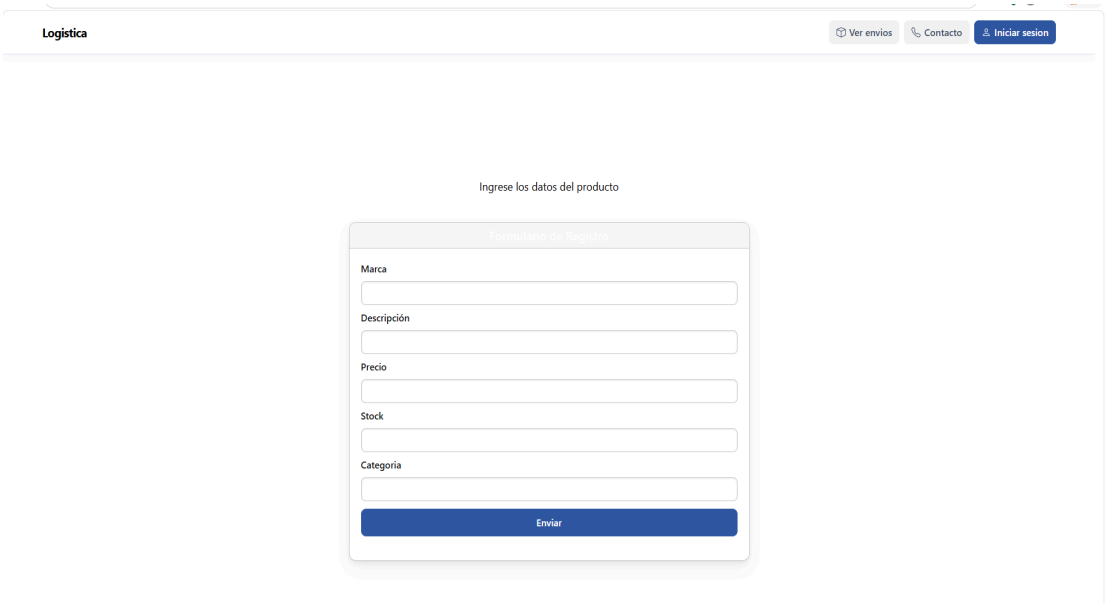
```
CREATE TRIGGER verificarExist
BEFORE INSERT ON Reportes_Ticket
FOR EACH ROW
BEGIN
   DECLARE v_existe_ticket INT;

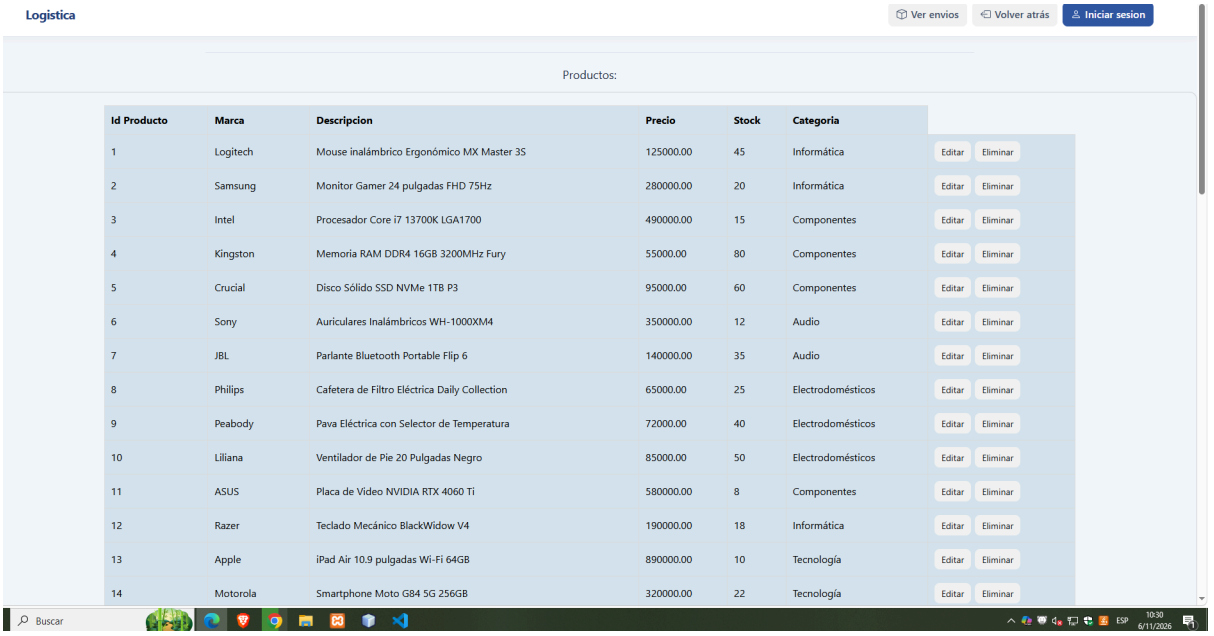
   SELECT COUNT(*) INTO v_existe_ticket
   FROM Tickets
   WHERE ID_Ticket = NEW.ID_Ticket;

   IF v_existe_ticket = 0 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Error: No se puede asociar el reporte porque el Ticket
indicado no existe.';
   END IF;
END//

DELIMITER ;
```

Despues comence con la conexión con el servidor web. Se me complicó mucho la parte de la conexión por lo tanto no llegue a terminarlo. Me tuve que juntar con mis compañeros Nicolás Rosato, Santiago Goldfarb y Benjamin para que me ayuden con eso. Estuve mucho tiempo con esto ya que no llegaba a entenderlo, también porque se me frizo la computadora varias veces.

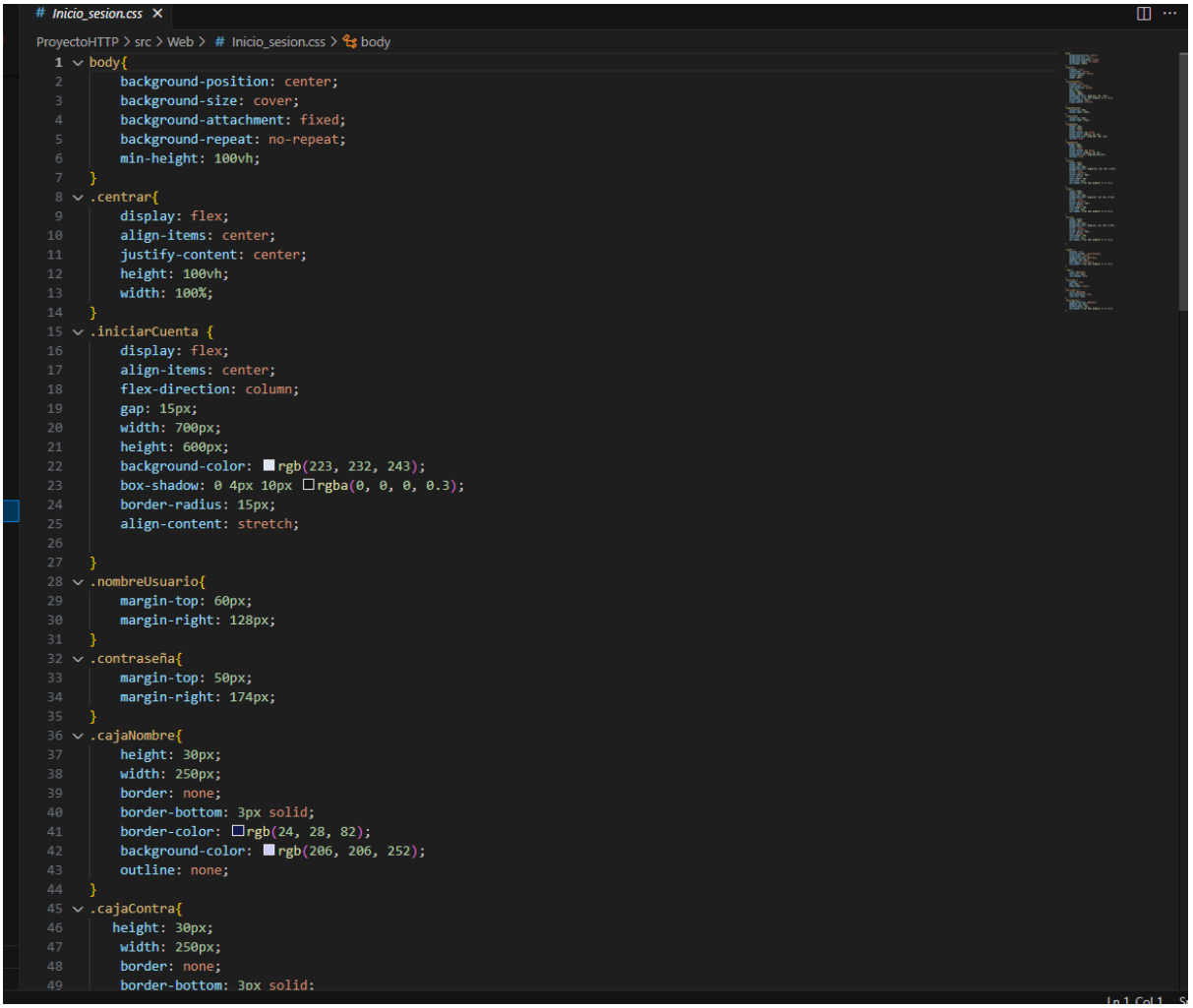




05/11/2026

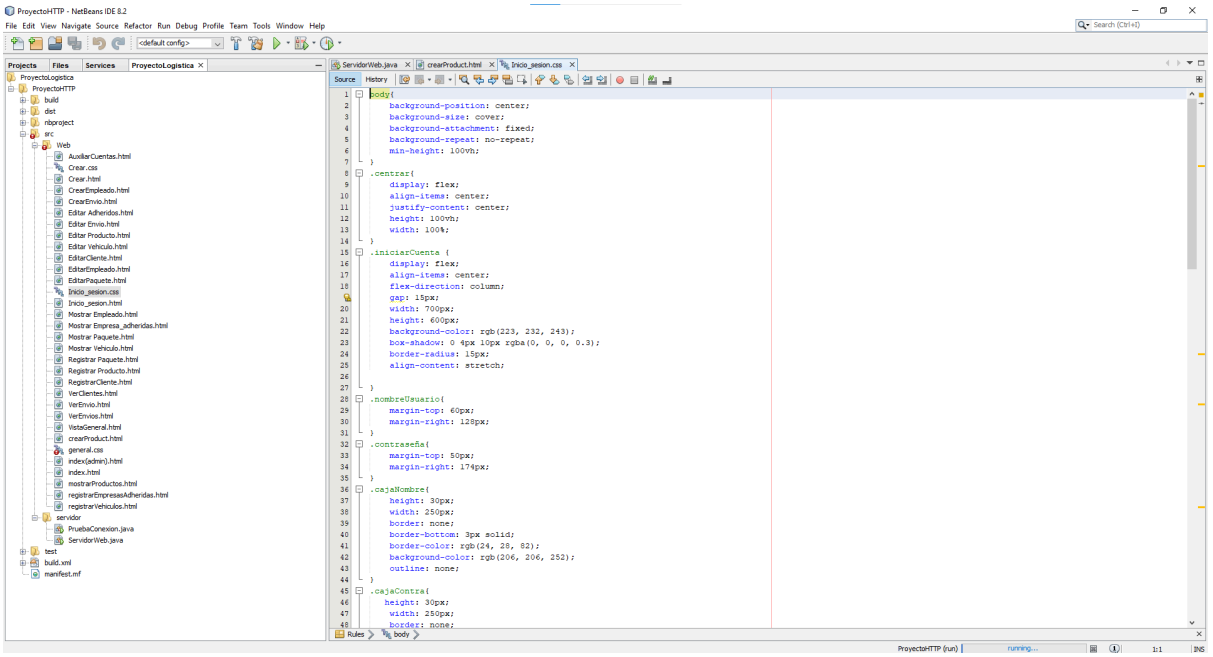
Finalmente hoy jueves 5 de noviembre terminé de realizar la conexión con el servidor web. También me puse a modificar el css de las páginas de inicio de sesión y la de crear ya que tenían colores que no iban con las páginas realizadas. Además

hoy nos juntamos todos los integrantes del grupo para ponernos de acuerdo que parte va a presentar cada uno. A mi me tocó presentar la parte de POO.



06/11/2026

Hoy es el día de la entrega, todavía nos faltan cosas por terminar. Entonces me puse a seguir con las actividades que me quedaban. Como ayer terminé de modificar el css, mis compañeros me pidieron que les pase lo que hice para así poder unirlo a la conexión de Html x Java x MySQL.


ProyectoHTTP - NetBeans IDE 8.2
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
Projects Files Services ProyectoLogistica
ProyectoLogistica
ProyectoHTTP
build
dist
nbproject
src
Web
AuxiliarCuentas.html
Crear.css
Crear.html
CrearEmpleado.html
CrearEnvio.html
Editar Adheridos.html
Editar Envio.html
Editar Producto.html
Editar Vehiculo.html
EditarCliente.html
EditarEmpleado.html
EditarPaquete.html
Inicio_sesion.css
Inicio_sesion.html
Mostrar Empleado.html
Mostrar Empresa_adheridas.html
Mostrar Paquete.html
Mostrar Vehiculo.html
Registrar Paquete.html
Registrar Producto.html
RegistrarCliente.html
VerClientes.html
VerEnvio.html
VerEnvios.html
VistaGeneral.html
crearProduct.html
general.css
index(admin).html
index.html
mostrarProductos.html
registrarEmpresasAdheridas.html
registrarVehiculos.html
servidor
PruebaConexion.java
ServidorWeb.java
test
build.xml
manifest.mf
ServidorWeb.java
crearProduct.html
Inicio_sesion.css
Source History
body{
background-position: center;
background-size: cover;
background-attachment: fixed;
background-repeat: no-repeat;
min-height: 100vh;
}
.centrar{
display: flex;
align-items: center;
justify-content: center;
height: 100vh;
width: 100%;
}
.iniciarCuenta {
display: flex;
align-items: center;
flex-direction: column;
gap: 15px;
width: 700px;
height: 600px;
background-color: rgb(223, 232, 243);
box-shadow: 0 4px 10px rgba(0, 0, 0, 0.3);
border-radius: 15px;
align-content: stretch;
}
.nombreUsuario{
margin-top: 60px;
margin-right: 128px;
}
.contraseña{
margin-top: 50px;
margin-right: 174px;
}
.cajaNombre{
height: 30px;
width: 250px;
border: none;
border-bottom: 3px solid;
border-color: rgb(24, 28, 82);
background-color: rgb(206, 206, 252);
outline: none;
}
.cajaContra{
height: 30px;
width: 250px;
border: none;
Rules body
ProyectoHTTP (run) running...
1:1 INS